MARTIAN MAP & PATHFINDING ENGINE

NASA Space Apps Challenge 2026 · Interplanetary Survival Guide: Martian Map · Jezero Crater

Technical Brief v2 — corrected 11 Oct 2026. Items marked **tested** were run in Python (pyproj, rasterio, scikit-image). Slope limits and speeds are **team assumptions**, not NASA specifications.

What changed from v1. (1) Code blocks now keep their line breaks. (2) `src.nodata`, not `src.nodatavalue`. (3) "10 m MOLA" removed: MOLA is 463 m/px; routing uses the 20 m CTX DEM. (4) DEM is reprojected to a local metric CRS before slope (pixel size in degrees is not metres). (5) Slope uses Horn's 3×3 method with real cell size, not `np.gradient` per pixel. (6) Cost table and formula now agree (the table *is* the function). (7) Distance is applied once: v1 applied it twice. (8) Routing uses scikit-image (4M cells in ~1.8 s) instead of a pure-Python loop (250k cells in ~5.7 s). (9) CRS table fixed (`IAU2000:49900` does not resolve; 49900 is a sphere). (10) Gale Crater dropped; Jezero is locked.

1. MISSION FOCUS & ARCHITECTURE

The challenge asks for a **layered, integrated map** that helps a future astronaut plan a Marswalk. The product is therefore a Jezero map with toggleable science layers and click-for-info; the route/hazard planner is the headline *feature* on top of it, not the whole project. Re-check the wording when full challenge statements are published on 28 Oct 2026.

Stage	What happens	Output
1 Acquire	Download DEM, imagery, minerals, thermal, weather for one bounding box	/data/raw + README
2 Clip & reproject	Clip all layers to the same box; reproject to a local metric CRS	/data/processed/*.tif
3 Derive	Slope, hillshade, hazard classes, cost grid (Section 4)	GeoTIFF + PNG overlays
4 Publish	Tile / PNG overlays, cost grid, MEDA time series as static files	/web/public/data
5 Display	Web map: layer toggles, opacity, click inspector, route tool, elevation profile	Live demo

Area of interest (proposal): 17.95–18.85 °N, 77.10–78.00 °E (planetocentric, east-positive), about 53 × 51 km. The crater is ~45 km wide and centred near 18.38 °N, 77.58 °E (verify). Clip to the DEM's real footprint with `gdalinfo`: the CTX DEM covers the whole crater, but full coverage of the box is unconfirmed.

2. MARS COORDINATE SYSTEMS

Planetocentric latitude (φ_c) is measured from the planet's centre; **planetographic latitude** (φ_g) from the surface normal. Mars is slightly flattened, so $\tan \varphi_c = (b/a)^2 \cdot \tan \varphi_g$, with a = 3396.19 km and b = 3376.20 km. **Tested:** at Jezero's latitude (18.4°) the two differ by ≈ 0.20°, about **12 km** on the ground — so mixing them really does misplace layers.

- Many NASA/USGS map products use planetocentric latitude with east-positive longitude, but **read each file's metadata** (`gdalinfo` or the PDS label) instead of assuming.
- MOLA heights are measured above the **areoid** (Mars's equivalent of sea level), not above a reference ellipsoid. State this in the README.
- 1° of latitude ≈ 59.27 km; 1° of longitude at 18.4 °N ≈ 56.24 km (tested).

Identifier	What it actually is	Use it for
ESRI:104971	Mars 2000 **sphere**, geographic (lat/lon), R = 3,396,190 m. Not a projection.	Default lat/lon for web display and clipping
ESRI:104905	GCS_Mars_2000, ellipsoidal geographic CRS	Only if a source file says ellipsoid / planetographic
IAU_2015:49900	Mars (2015) sphere, planetocentric. Resolves in pyproj. (`IAU2000:49900` from v1 fails.)	Alternative to ESRI:104971
Local metric PROJ string	`+proj=eqc +lat_ts=18.4 +lat_0=0 +lon_0=77.5 +a=3396190 +b=3396190 +units=m +no_defs`	**Slope, distance and routing**

Rules: EPSG codes are Earth-only, so pass ESRI/IAU codes or PROJ strings, and test every code with `pyproj.CRS.from_user_input(...)` before relying on it. A polar-stereographic CRS (v1) is wrong for a site at 18 °N. The HiRISE DEM mosaic's filename shows it is equirectangular with lat_ts = 0, so its east-west pixel spacing is ~5 % larger than ground metres at 18 °N. Always reproject before computing slope.

3. DEM INGESTION

`abs(transform[0])` is metres *only* for projected rasters. A geographic DEM returns degrees (tested: 0.0017 instead of ≈ 100 m). So the DEM is reprojected first:

```
import numpy as np, rasterio
from rasterio.warp import calculate_default_transform, reproject, Resampling

LOCAL_CRS = ("+proj=eqc +lat_ts=18.4 +lat_0=0 +lon_0=77.5 "
             "+a=3396190 +b=3396190 +units=m +no_defs")

def load_dem(path, res_m=None):
    """Read DEM, mask NoData as NaN, reproject to LOCAL_CRS. Returns (dem, metres/pixel)."""
    with rasterio.open(path) as src:
        tr, w, h = calculate_default_transform(
            src.crs, LOCAL_CRS, src.width, src.height, *src.bounds, resolution=res_m)
        dem = np.full((h, w), np.nan, dtype=np.float32)
        reproject(rasterio.band(src, 1), dem,
                  src_nodata=src.nodata, dst_nodata=np.nan,       # src.nodata, NOT nodatavalue
                  dst_transform=tr, dst_crs=LOCAL_CRS, resampling=Resampling.bilinear)
    return dem, abs(tr.a)
```

4. SLOPE & COST SURFACE

Slope is scale-dependent. A 463 m MOLA grid averages steep ground away, so a 15° limit would almost never trigger. Use the **20 m CTX DEM** for the route and the 1 m HiRISE DEM only for local close-ups. Slope uses Horn's 3×3 method (the one `gdaldem slope` uses): $dz/dx = [(c + 2f + i) − (a + 2d + g)] / (8·\Delta x)$, likewise for dz/dy, and $\theta = \arctan(\sqrt{(dz/dx)^2 + (dz/dy)^2})$ in degrees, with Δ in **metres**.

Cost. Each cell gets a multiplier m(θ) from the table below by linear interpolation. Optional extra layers (roughness, shadow, comms) are added to m. The router then multiplies by the step length (1 or √2) **once**. v1's formula gave 4.2× at 5°, 19× at 10° and 51× at 15°, contradicting its own table; a lookup table cannot disagree with itself.

Slope band	Rating	Multiplier (start → end of band)	Meaning (assumption)
0° – 5°	Optimal	1.0× → 2.5×	Flat to gentle; normal walking pace
5° – 10°	Moderate	2.5× → 8.0×	Noticeably more effort and oxygen use
10° – 15°	High hazard	8.0× → 35×	Steep; slip risk; only if no alternative
> 15°	Blocked	∞ (np.inf)	Treated as impassable; expose as a UI slider

```
def slope_deg(dem, res_m):                      # Horn 3x3; NaN in -> NaN out
    z = np.pad(dem, 1, mode="edge")
    a, b, c = z[:-2, :-2], z[:-2, 1:-1], z[:-2, 2:]
    d,    f = z[1:-1, :-2],              z[1:-1, 2:]
    g, h, i = z[2:, :-2],  z[2:, 1:-1],   z[2:, 2:]
    dzdx = ((c + 2*f + i) - (a + 2*d + g)) / (8 * res_m)
    dzdy = ((g + 2*h + i) - (a + 2*b + c)) / (8 * res_m)
    return np.degrees(np.arctan(np.hypot(dzdx, dzdy)))

SLOPE_ANCHORS = [0.0, 5.0, 10.0, 15.0]          # degrees  (team assumptions: tune them)
COST_ANCHORS  = [1.0, 2.5, 8.0, 35.0]           # cost multipliers

def cost_surface(slope, limit_deg=15.0, extra=None):
    cost = np.interp(slope, SLOPE_ANCHORS, COST_ANCHORS).astype(np.float64)
    if extra is not None:
        cost = cost + extra                     # roughness, shadow, comms ...
    cost[~np.isfinite(slope) | (slope > limit_deg)] = np.inf
    return cost
```

Not verified: the 15° limit and v1's "~4 km/h EVA speed" could not be confirmed as official figures. Keep them as named, adjustable parameters, label them "assumption" in the UI, and replace them if a citable source is found.

5. ROUTING ENGINE

Approach	Measured (sandbox)	Verdict
scikit-image `route_through_array`	2000 × 2000 (4M cells): 1.8 s	**Use this.** C speed, handles np.inf as blocked, returns the path
Pure-Python heapq A* (v1 style)	500 × 500 (250k cells): 5.7 s	Too slow; v1 also double-counted distance and returned no path
Numba A*	not benchmarked	Only if skimage proves too slow after downsampling
JS router in the browser	not tested	Option: ship the cost grid, route offline (no server to fail)

- Plan on a **40–60 m grid** (≈ 900–1300 px per side across the box) or a window around start and goal. “Sub-second re-routing” on the full-resolution grid is not realistic in Python.
- skimage’s geometric MCP is a Dijkstra-type search (not A*) and already weights steps by length: **do not multiply by distance again**.
- If you write A* yourself: apply cost once, use heuristic = Euclidean distance × minimum cell cost (≥ 1, so it stays admissible), skip stale heap entries, and reconstruct the path from the parent map.
- Known limits: cost is the same uphill and downhill; rocks and dust are ignored unless added as layers; 8-connected paths zig-zag slightly (smooth before display).

```
from skimage.graph import route_through_array
path, total_cost = route_through_array(cost, start=(r0, c0), end=(r1, c1),
                                       fully_connected=True, geometric=True)
# path = [(row, col), ...]; length_m = sum(step lengths) * res_m
```

A tested reference file, `mars_route_core.py`, implements Sections 3–5 end to end and includes a synthetic ridge-with-a-pass demo: the route found the only pass (5 % of cells blocked).

6. MVP DATASETS (VERIFIED SOURCES)

Layer	Product	Resolution	Where
Elevation	Mars 2020 CTX DEM mosaic	20 m/px; whole Jezero crater	USGS Astrogeology / planetarymaps.usgs.gov (M20_JezeroCrater_CTXDEM_20m.tif)
Imagery	CTX orthomosaic matched to the DEM	5 m/px	USGS Astropedia (M20_JezeroCrater_CTXortho_mosaic_5m.tif)
Context DEM	MOLA global DEM	463 m/px	USGS: Mars_MGS_MOLA_DEM_mosaic_global_463m.tif
Minerals	CRISM-based Jezero mineral map (jezeroFinal.tif)	see README in record	Zenodo record 5575824; partial coverage around the landing ellipse
Thermal	THEMIS day/night IR global mosaic	100 m/px	USGS Astropedia / Mars Trek (exact URL to confirm)
Weather	MEDA derived/calibrated CSV, one file per sensor per sol	time series	PDS Atmospheres Node (pds-atmospheres.nmsu.edu, Perseverance → MEDA)
Optional zoom	HiRISE DTM 1 m + ortho 25 cm	1 m / 0.25 m	USGS; GB-scale, landing-site only. Skip for the MVP.

Record source, resolution, CRS and download date for every layer in the README. MEDA is a time series at the rover’s position, not a map: show it as a chart panel. Check file sizes before downloading.

7. ROADMAP & ROLES (UPDATED)

Event: **14–15 Nov 2026**. Full challenge statements and data sets: **28 Oct**. Judging starts 16 Nov. Confirm the 2026 rules on pre-built code before writing project code early.

Team	Who	Responsibilities
Frontend & visualisation	Talha, Rafay	Map shell and layer toggles, click inspector, route tool UI, elevation profile and MEDA charts, HUD styling, pitch and video
Data & algorithms	Selenophile, Maheen	Acquire and clip layers, reproject, slope/hazard/cost layers, tiles/PNGs, routing, hand-off files and README

When	Milestone
by 17 Oct	Bounding box, MVP layer list and README template locked; DEM clipped and handed to the data pipeline first
18–27 Oct	All layers in /data/processed; slope/cost and routing running on the real DEM; frontend shell runs on placeholder data; data contract agreed
28 Oct – 2 Nov	Read the official statement; cut or add features to match
3–13 Nov	Full timed dry run: data → map → route → pitch; fix the weakest link
14–15 Nov	Sat: real data in the map, working route. Sun: freeze features, record video, polish, submit

Open items: official slope/speed limits; DEM footprint vs bounding box; THEMIS download URL; whether pre-built code is allowed; who owns routing now that the team split changed.